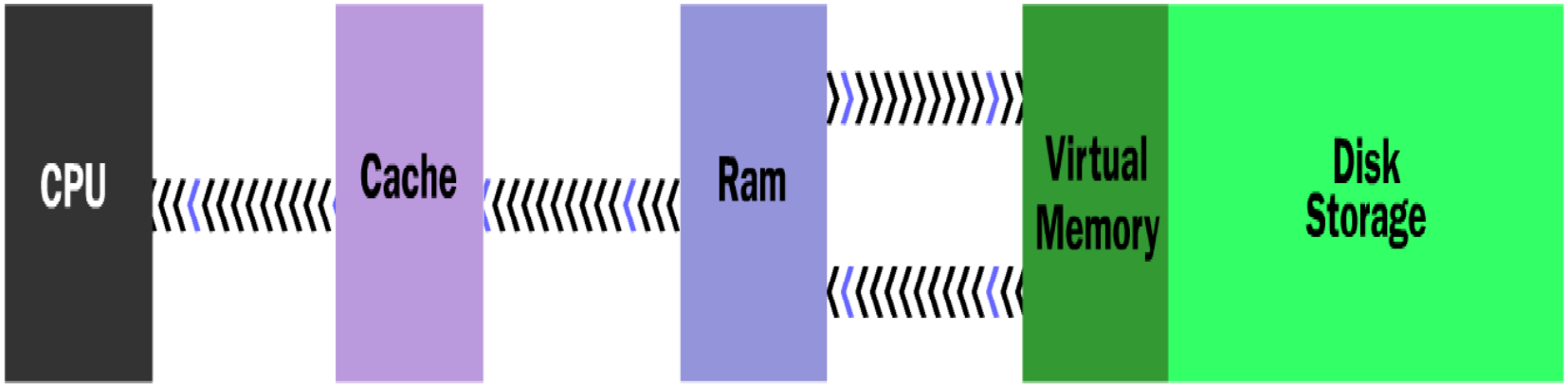
İşletim Sistemleri
(Operating Systems)



SANAL BELLEK (Virtual Memory)

Sanal Bellek (Virtual Memory)

- Sanal bellek **(virtual memory)**, ikincil belleğin fiziksel ana belleğin bir parçası gibi kullanılabileceği bir bellek yönetimi tekniğidir.
- Bilgisayarlar sınırlı miktarda fiziksel ana belleğe sahiptir, bu nedenle aynı anda birden fazla process çalıştığında ana bellekte boş alan tükenme noktasına gelir. Sanal bellek kullanan bir sistem, ana belleği taklit etmek için sabit diskin bir bölümünü kullanır.
- Daha yavaş olan sanal bellek, yalnızca ana bellek doldurulduğunda kullanılır.
- Fiziksel ana belleğin yetersiz kaldığı durumlarda, işletim sistemi yakın zamanda kullanılmayan sayfaları (page) veya kesimleri (segment) sabit disk gibi bir depolama aygıtına göndererek ana bellekte yer açar.

Sanal Bellek (Virtual Memory)

- **Sanal Bellek Avantajları:**

- Sanal bellek, sistem performansını iyileştirmek , çoklu process yürütmesini sağlamak ve büyük programları kullanmak için önemlidir

- Programlar fiziksel bellekten büyük olduğunda, sanal bellek kullanımı ile, kullanıcı fiziksel ana bellek kısıtlamalarından kurtulmaktadır.

- Fiziksel belleğin process'ler arasında etkin ve güvenli bir şekilde paylaşılması sağlanır.

Sanal Bellek (Virtual Memory)

- **Sanal Bellek Avantajları:**

- Sanal bellek (virtual memory) yöntemi, bir process'in tamamının fiziksel belleğe yüklenmeden çalıştırılmasına izin verir. Process'in sadece gerekli bölümleri belleğe yüklenir.

- Bu sayede fiziksel bellekte daha fazla process bulunabilir. Böylece çok sayıda process eşzamanlı olarak çalıştırılabilir. Yani çoklu programlamayı daha etkin kılar.

Sanal Bellek (Virtual Memory)

- **Sanal Bellek Avantajları:**

- Sanal bellek, programcı için fiziksel bellek alanı limitini ortadan kaldırır.

- Sanal bellek, programcıya fiziksel ana bellekten daha büyük bir alan sağlar ve programların boyutu fiziksel ana belleğin boyutundan bağımsızlaşır.

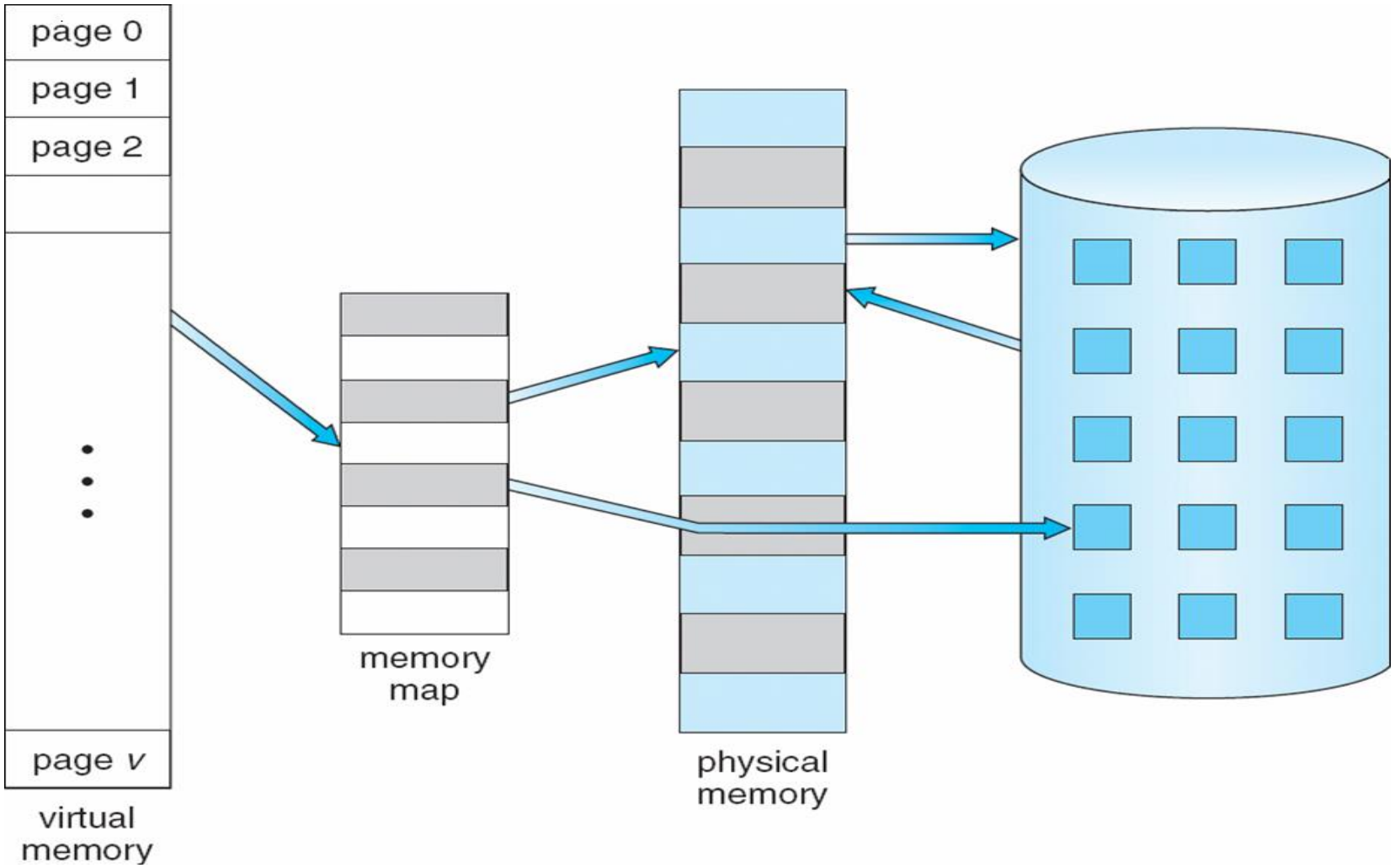
Sanal Bellek (Virtual Memory)

- **Sanal Bellek Dezavantajları:**

- İşlemci zamanı, process parçalarını fiziksel bellek ve sanal bellek arasında taşımakla geçer.
- Ancak, sanal belleği fiziksel belleğe değiştirme işlemi oldukça yavaştır. Bilgisayarın fiziksel bellek erişimi sanal bellek erişimine göre çok daha hızlıdır.
- Sanal belleğin kullanılması genellikle performansta gözle görülür bir düşüşe neden olur.

- Bu nedenle büyük kapasiteli fiziksel belleğe sahip olmak her zaman en iyi çözümdür.

Sanal bellek, programcıya fiziksel hafızadan daha büyük bir alan sağlar.



Sanal Bellek (Virtual Memory)

- 1980’lerin DOS’u veya 1960’ların anabilgisayar işletim sistemleri gibi daha eski işletim sistemlerinde bu işlev bulunmamaktadır.

- Gömülü sistemlerde ve bazı özel amaçlı bilgisayar sistemlerinde hızlı ve uyumlu tepki zamanı gerektiğinden genellikle sanal bellek kullanılmaz.

Sanal Bellek (Virtual Memory)

- Sanal bellek kullanan işletim sistemlerinde, process'ler yürütümleri sürecinde bellek ile disk arasında sürekli yer değiştirir.
- Process'lerin disk ile bellek arasında yer değiştirilmesinin nedeni, yürütülmesi gereken process'ler için fiziksel bellekte yeterli alan bulunmamasıdır.
- Bu durumda bellekte yer açmak üzere, yürütülmesi tamamlanmamış bir process geçici olarak, bu amaçla öngörülen disk alanlarına (sanal belleğe) taşınabilir.

Sanal Bellek (Virtual Memory)

- Sanal bellek yönetiminin uygulandığı sistemlerde, yürütülmeye alınan process'lere, tüm mantıksal adres uzayını karşılayacak kapasitede fiziksel ana bellek alanı yerine disk alanı atanır.

- Birçok durumda process'in tamamının bellekte bulunması gerekmemektedir.

- Process'in sadece bazı kısımlarına farklı zamanlarda ihtiyaç duyulabilmektedir. Yalnızca o anda ihtiyaç duyulan process bölümü ana belleğe aktarılabilir.

Sanal Bellek (Virtual Memory)

- Bir process yürütümü başlatılırken ilk sayfaların ya da ilk segmentin ana bellekte bulunmasıyla yetinilir.
- Herhangi bir zaman diliminde bir process'in ana bellekte bulunan kısmına yerleşik set (resident set) denir.
- Ancak, gereksinim duyulup da ana bellekte bulunmayan sayfa yada segmentlerin ana belleğe nasıl taşınacağı sorusunun cevaplanması da gereklidir.
- Bu yüzden, gereksinim duyulup da ana bellekte bulunmayan sayfa ya da segmentlerin belleğe taşınma işlemleri de yönetilmelidir.

Sayfalı Sanal Bellek Yönetimi

- Sayfalı sanal bellek yönetiminde, amaç programlar (object program) derleyiciler tarafından eşit uzunlukta sayfalar biçiminde hazırlanırlar.
- Sanal bellek düzeni çerçevesinde, hazırlanan bu programların ana belleğe yüklenmeye hazır kopyaları, tümüyle diskte oluşturulur.
- Diskte yer alan sayfalardan bir bölümü fiziksel belleğe yüklenerek process yürütümü başlatılır.

Sayfaların Belleğe Getirilme Biçimi

- **İstemsel Sayfalama (Demand Paging) Tekniği :**

- Sanal bellek sistemleri genellikle **istemsel sayfalama (demand paging)** tekniğini kullanır.
- **İstemsel sayfalama, process'lere ait sayfaların ihtiyaç olduğunda ana belleğe yüklenmesidir.**
- İstemsel sayfalama ile process çalışması süresince kullanılmayan sayfalar fiziksel ana belleğe yüklenmez.
- Başlangıçta sayfa hataları oluşacak fakat zamanla bu hatalar azalacaktır.

Sayfaların Belleğe Getirilme Biçimi

- **İstemsel Sayfalama (Demand Paging) Tekniği :**

- Yürütülmek istenen uygulamalar genelde ikincil bellekte (çoğunlukla diskte) bulunur ve yürütülmek üzere fiziksel ana belleğe taşınır.

- Ana belleğe yerleştirilen bir process'in hangi sayfasının kullanılacağına **Sayfalayıcı (Pager)** *('lazy swapper' olarak da adlandırılır)* karar verir.

Sayfaların Belleğe Getirilme Biçimi

- **İstemsel Sayfalama (Demand Paging) Tekniği :**

- Sayfalayıcı (Pager) fiziksel belleğe process'in kullanılacak olan sayfasını getirir. Böylece sayfalayıcı kullanılmayacak olan sayfaları fiziksel belleğe getirmez.

- Bir sayfanın ihtiyaç duyulmadığı sürece belleğe alınmamasına pure demand paging denir.

Swapper ve Pager

- **Sayfalı Sanal Bellek yönetiminde yer değiştirici (swapper) yerine sayfalayıcı (pager) kullanımı daha doğru olacaktır.**

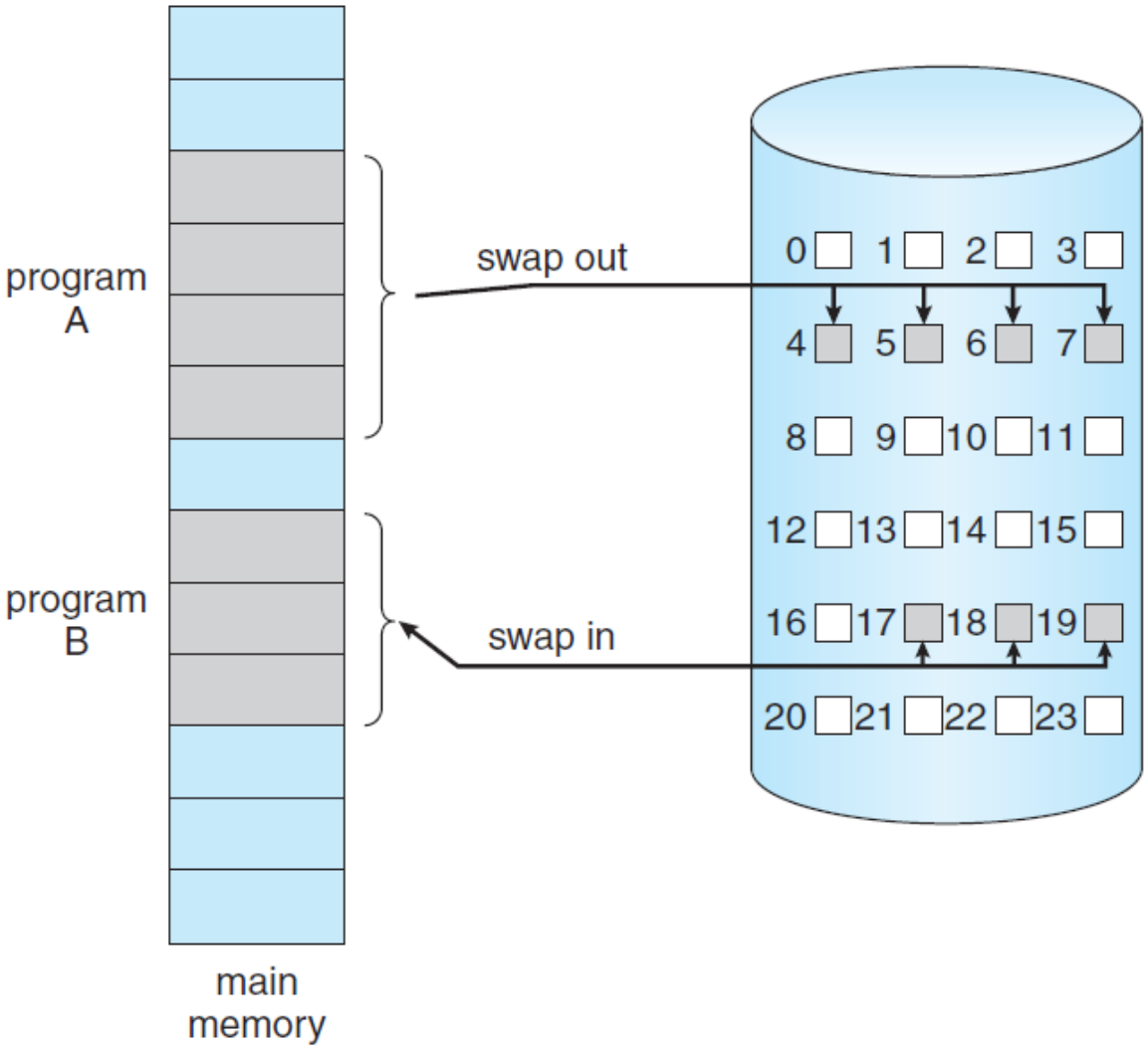
- **Yer değiştirici (swapper) process'in bütünü ile ilgilenir.**

- **Sayfalayıcı (pager) sadece gerekli olan sayfa ile ilgilenir.**

Sayfaların Belleğe Getirilme Biçimi

swap in : Process sayfalarının diskten fiziksel ana belleğe getirilmesi
swap out : Process sayfalarının fiziksel ana bellekten diske taşınması

Tüm sayfalar yerine sadece yürütülecek olan sayfaların ana belleğe getirilmesi yer değiştirme zamanını (swap time) düşürdüğü gibi, ana bellekte daha az yer kullanılmasını sağlar.



Sayfaların Belleğe Getirilme Biçimi

- Fiziksel belleği 32 MB olan bir bilgisayarda, 300 MB'lık bir program çalıştırılmak istenirse, bu programın tümünün fiziksel belleğe yerleşmesi mümkün değildir.
- Bu durumda istemsel sayfalama (demand paging) yöntemi kullanılarak, örneğin fiziksel bellek 4 KB'lık sayfalara bölünerek, çalıştırılmak istenen programın sadece gerek duyulan sayfaları fiziksel belleğe yüklenir.
- Geri kalanı ise sanal bellekte bekler ve gerekli olduğunda fiziksel belleğe getirilir.

Sayfaların Belleğe Getirilme Biçimi

- Sanal bellek düzeninde tüm process sayfalarının fiziksel bellekte bulunma zorunluğu olmadığından sayfa tablolarında, ilgili sayfanın bellekte yüklü olup olmadığını gösteren bir **durum belirteci (valid – invalid bit)** kullanılır.

- Durum belirteci hangi sayfaların fiziksel ana bellekte hangilerinin ikincil bellek (disk) üzerinde olduğunu ayırt etmek için kullanılır.

Sayfa tablosu girdileri (Page Table Entries)

- **Modify bit (M) :** Sayfa üzerinde yazma yapılıp yapılmadığını belirtir. Sayfaya bir yazma yapıldığında M biti 1 olur.
- **Reference bit (R) :** Sayfaya erişilip erişilmediğini belirtir. Sayfaya erişildiğinde yani bir okuma veya yazma gerçekleştiğinde R biti 1 olur.
- **Valid/invalid bit (V) :** **Durum belirteci** olarak bilinir. Sayfa tablosunda, ilgili sayfanın bellekte yüklü olup olmadığını gösterir . Yani hangi sayfaların ana bellekte hangilerinin ikincil bellek (disk) üzerinde olduğunu ayırt etmek için kullanılır. valid:1, invalid:0
- **Protection bits (Prot) :** Sayfada hangi işlemlere izin verildiğini belirtir. Oku (read), yaz (write), yürüt (execute).
- **Page frame number :** Fiziksel bellekteki çerçeve (frame) numarasını gösterir.

1	1	1	2	20
M	R	V	Prot	Page Frame Number

- Bu bitler her bellek başvurusunda güncellenmelidir, bu yüzden donanım ile yapılmalıdır.

Sayfaların Belleğe Getirilme Biçimi

- **durum belirteci**, bir bit uzunluğunda bir bilgidir, **sayfa bellekte biti** olarak da bilinir.

- **Valid - Invalid** şeklinde bir bitlik durum belirteci, sayfanın bulunduğu yeri (ana bellekte olup olmadığı) belirlemek için kullanılır.

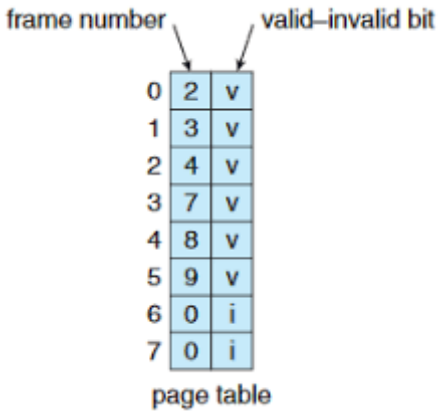
- Durum belirteci konumu

❖ **Valid (mantıksal 1)**

❖ **Invalid (mantıksal 0)**

Sayfaların Belleğe Getirilme Biçimi

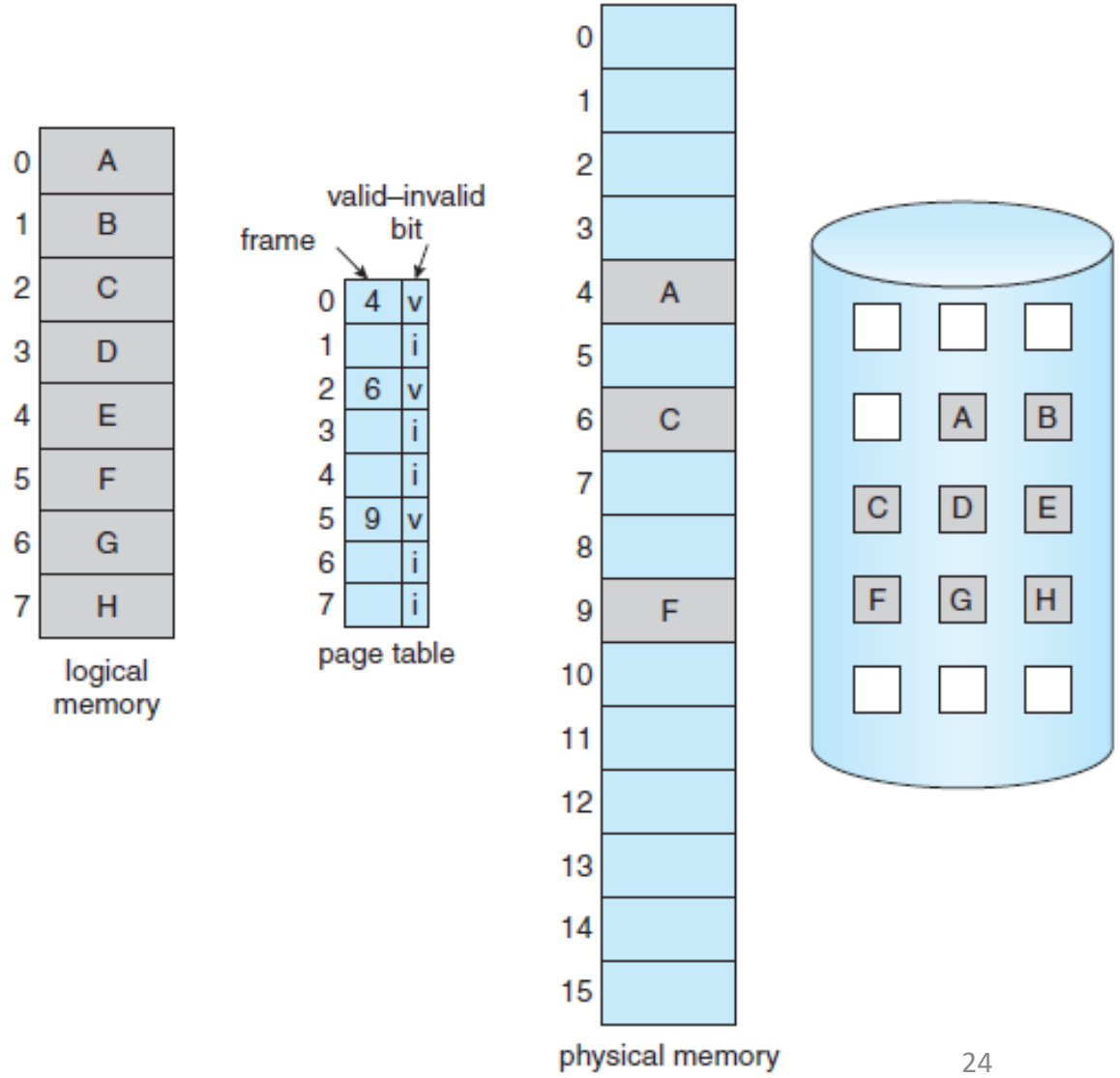
- Durum belirteç biti, **valid** konumda bırakılırsa bu değer ilişkili olduğu sayfanın geçerli ve fiziksel ana bellekte olduğunu gösterir.
- Eğer durum belirteç biti **invalid** durumda bırakılırsa ait olduğu sayfanın geçersiz olduğunu ve o an fiziksel ana bellekte olmadığını, diskte bulunduğunu gösterir.



Sayfaların Belleğe Getirilme Biçimi

- Durum belirteç biti
- (valid - invalid bit)

- Eğer process fiziksel bellekte olmayan bir sayfayı kullanmak isterse, bu girişim işletim sistemine "sayfa hatası (page fault)" olarak geri dönmektedir.



Sayfaların Belleğe Getirilme Biçimi

- **Sayfa hatası (page fault) :**
- Process yürütümü esnasında process'e ait bir sayfaya ihtiyaç olduğunda, fiziksel adres eşlemesi amacıyla process'in sayfa tablosunun erişilecek sayfayla ilgili satırına erişilir,

- İlk olarak sayfa tablosunun ilgili satırındaki **durum belirteç biti** donanımsal bellek yönetim birimince (Memory Management Unit-MMU) test edilir.

- MMU, mantıksal bir adres alanını karşılık gelen bir fiziksel adrese donanımsal olarak eşleyebilir.

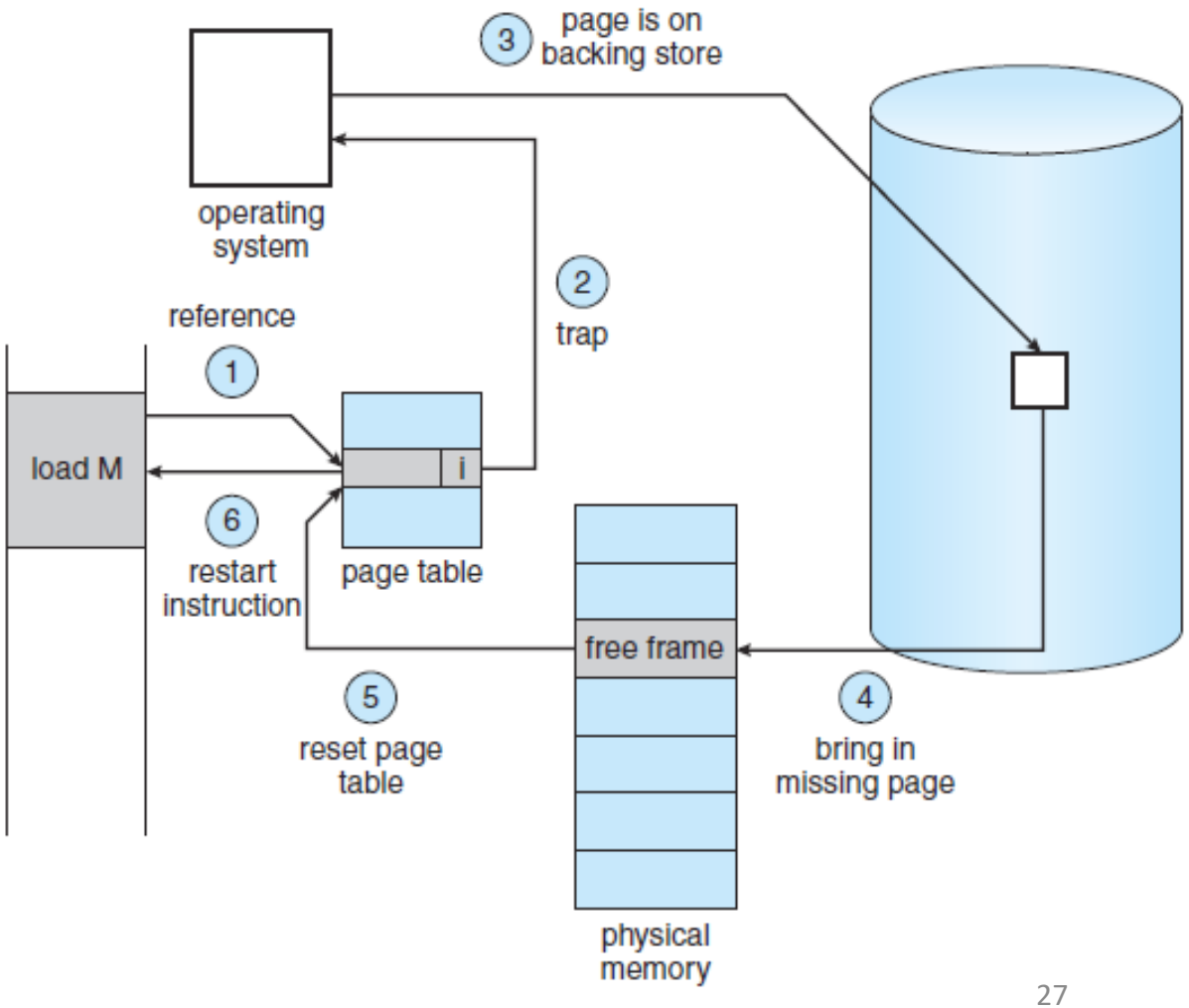
Sayfaların Belleğe Getirilme Biçimi

- **Sayfa hatası (page fault) :**

- İlgili sayfanın fiziksel ana bellekte yüklü olmadığı anlaşılırsa, yani durum belirteç biti **invalid** ise **sayfa hatası (page fault)** oluşur ve adres eşleme işlemleri (mantıksaldan fiziksel adrese eşleme) yarıda kesilerek denetim <u>işletim sistemi bellek yöneticisine</u> bırakılır.
- Bu, bir makina komutunun, işletimin ortasında kesilmesi demektir.
- İşletim sistemi bellek yöneticisi, fiziksel ve sanal bellek arasındaki geçişleri takip etmekten sorumludur.

Sayfaların Belleğe Getirilme Biçimi

1. Bir process'e ait bir sayfa talep edildiğinde, process'e ait sayfa tablosundaki durum belirteç biti MMU tarafından test edilir.
2. Bellekte bulunmayan bir sayfaya erişim söz konusu olduğunda, MMU işletim sistemine uyarı verir (trap). MMU işletim sisteminin bellek yöneticisine anahtarlanır.
3. İşletim sistemi ilgili sayfayı sanal bellekten fiziksel ana belleğe aktarmalıdır.
4. Fiziksel Bellekte ilgili sayfaya atanacak boş bir çerçeve (frame) bulunup bulunmadığı sınanır.
5. Fiziksel Bellekte boş bir çerçeve (frame) bulunabilirse bu sayfanın yerleştiği çerçeve numarası, process'in sayfa tablosuna işlenir.
6. Sayfa tablosundaki, erişilen sayfayla ilgili belirteçler de güncellenerek yarıda kesilen komutun işletiminin başına geri dönülür.

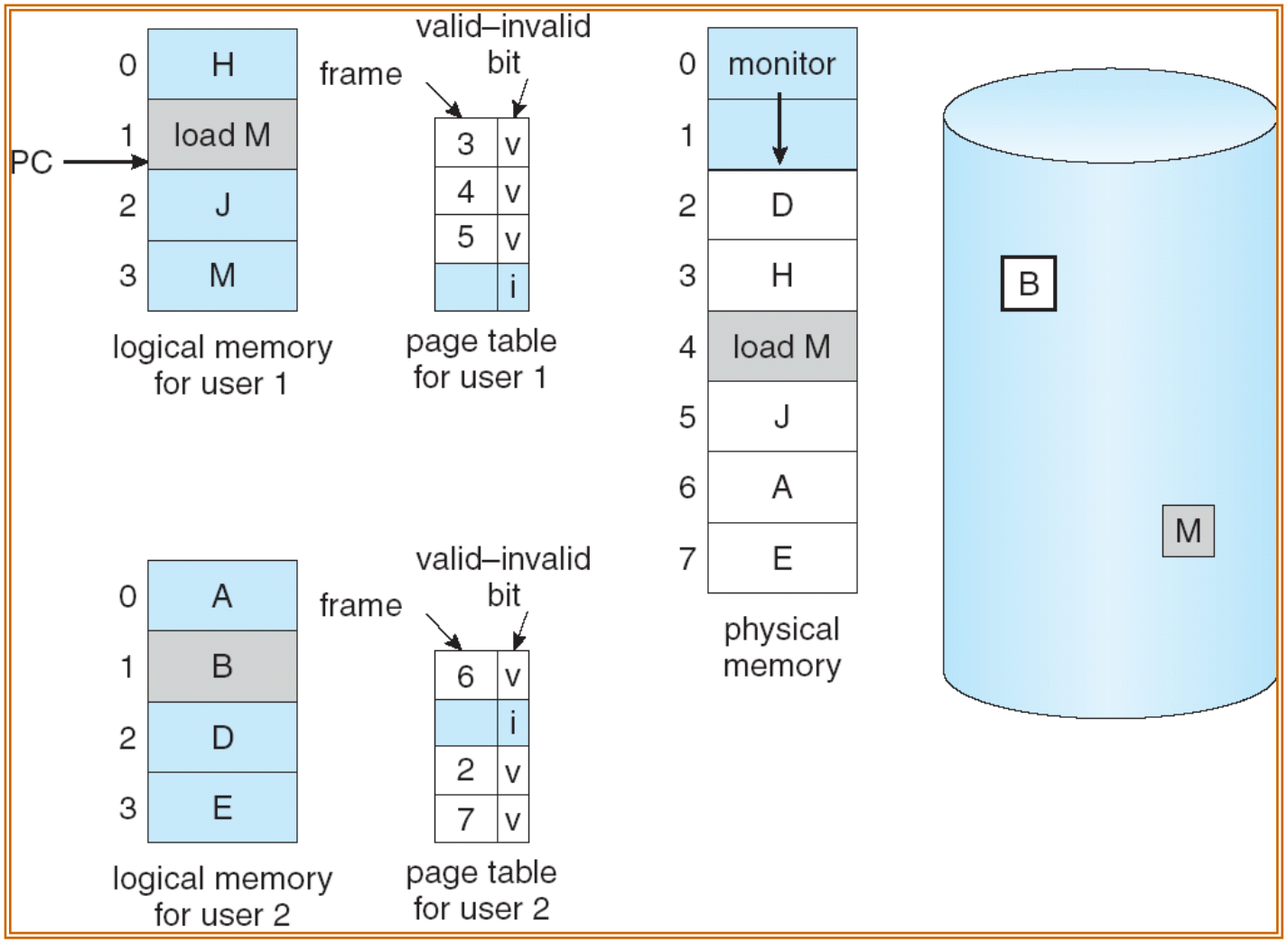


Sayfaların Belleğe Getirilme Biçimi

- mantıksaldan fiziksel adrese eşleme sürecinin ortasında, denetimin donanımdan işletim sistemi bellek yöneticisine aktarılmasından sonra gerçekleşmesi gereken işlemler şunlardır:

- İstenilen ilgili sayfa sanal bellek (sabit disk) üzerinde bulunur.
- Fiziksel ana bellek üzerinde ilgili sayfanın atanacağı boş çerçeve (frame) aranır.
  - Eğer ana bellekte boş çerçeve (frame) varsa o kullanılır.
  - Eğer ana bellekte boş çerçeve (frame) yoksa, **sayfa yerleştirme algoritmalarına göre (page replacement algorithm)** bir **kurban çerçeve (victim frame)** seçilir.
  - Seçilen kurban çerçevedeki sayfa (diskteki kopyasından farklı ise) sanal bellek üzerindeki yerine tekrar yazılır.
- İstenilen ilgili sayfa, boşaltılan çerçeve içine yerleştirilir.
- Sayfa tablosu değiştirilerek ilgili sayfanın belleğe yüklendiği belirtilir (durum belirteç biti **valid** olarak güncellenir).
- Sayfa hatası nedeniyle yarıda kesilen komutun işletimi yeniden başlatılır (restart instruction).

Bellekte boş çerçeve (frame) bulunamaması



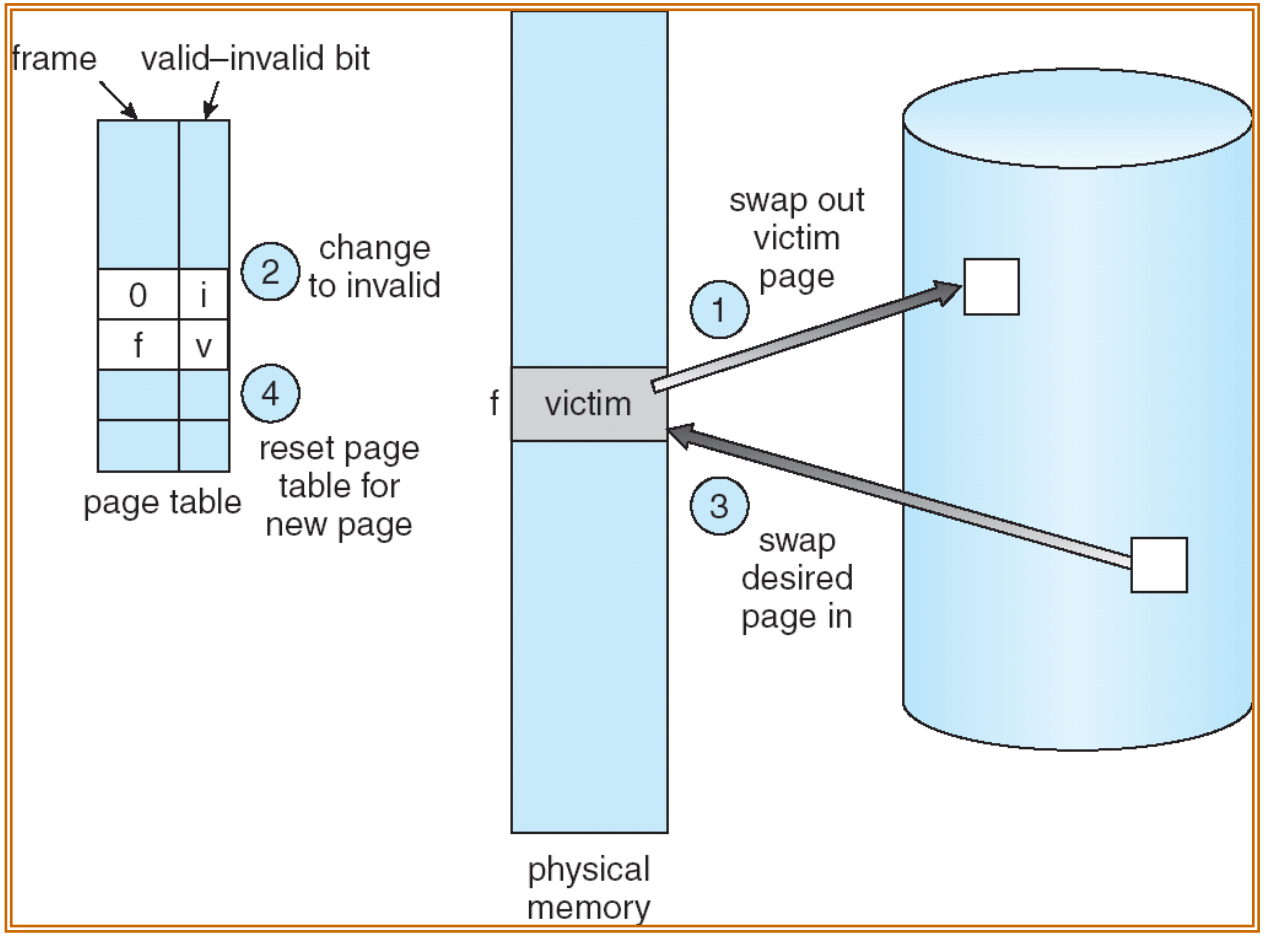
Sayfa B için fiziksel bellekte boş çerçeve bulunamaması durumu

Bellekte boş çerçeve (frame) bulunamaması

Eğer fiziksel ana bellekte boş çerçeve (frame) yoksa, **sayfa yerleştirme algoritmalarına** göre bir kurban çerçeve (victim frame) seçilir.

Seçilen kurban (victim) çerçevedeki sayfa (diskteki kopyasından farklı ise) sanal bellek üzerindeki yerine tekrar yazılır ve sayfa tablosunda gerekli değişiklik yapılarak o sayfanın artık ana bellekte olmadığı belirtilir.

Boşaltılan çerçeve artık istenilen sayfa için kullanılabilir durumdadır.



Modify bit

- Seçilen kurban (victim) çerçevedeki sayfa diskteki kopyasından farklı ise bu duruma da dikkat edilmelidir.
- Bu durum sayfa tablosundaki **modify bit** kullanılarak ayarlanabilir.
- Sayfa içeriği fiziksel belleğe alındıktan sonra **modify bit** sıfırlanır.
- Daha sonra bu sayfa üzerinde herhangi bir değişiklik olursa bu bit dirty modified olarak set edilir.
- Sayfa değiştirme sırasında modify bit incelenir.
- Eğer sayfa içeriğinde herhangi bir değişiklik varsa (dirty modified) o zaman sayfa sanal belleğe yazılır.
- Eğer sayfa içeriğinde bir değişiklik yoksa sayfanın tekrar sanal belleğe yazılmasına gerek kalmaz.

Bellekte Çerçeve (frame) Ataması

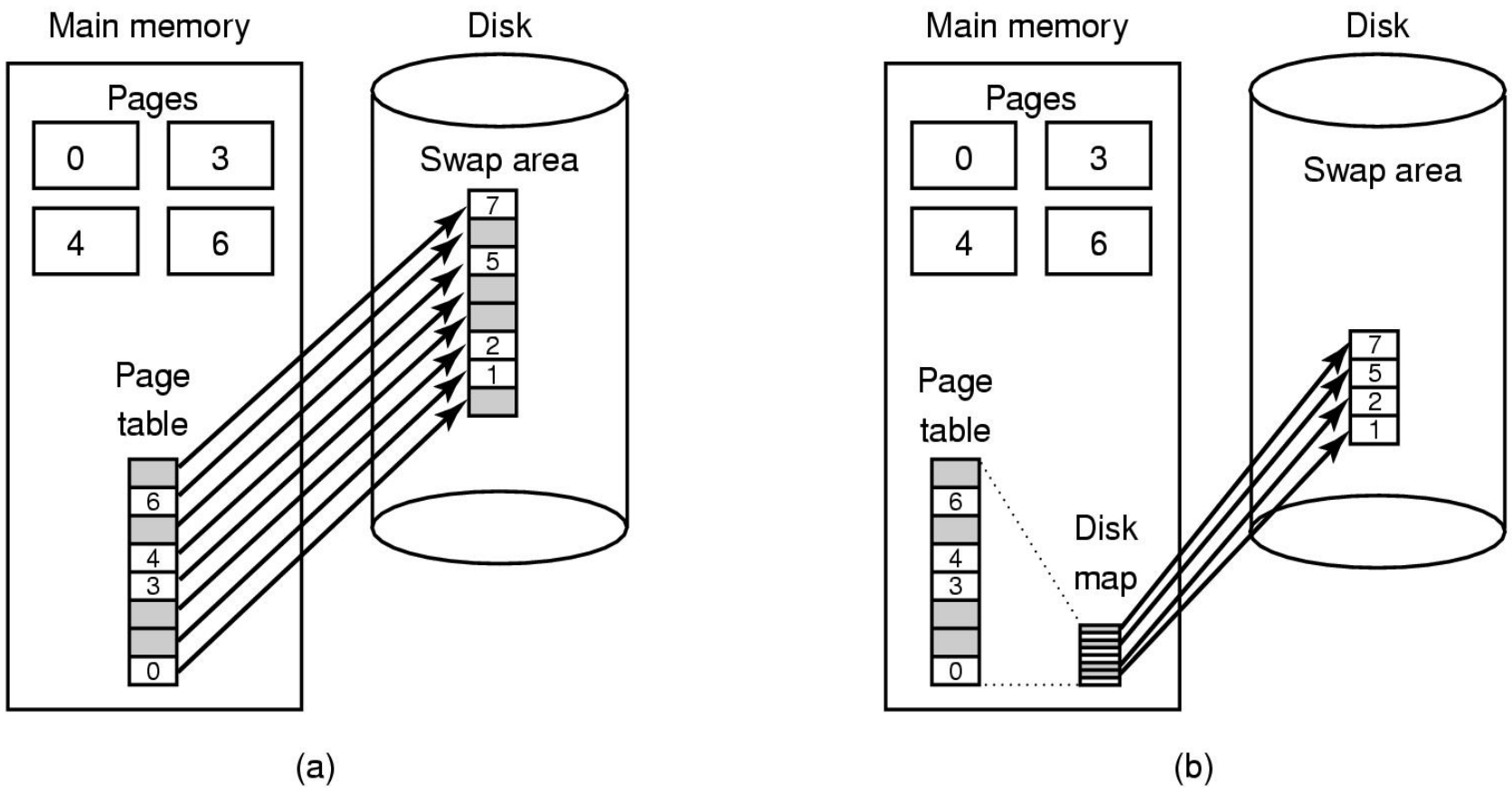
- **Statik yer atama:**
- Process'lere önceden belirlenen sayıda fiziksel bellekte çerçeve atanır. Yani bir process'e fiziksel bellekte ayrılabilecek çerçeve sayısı, işletimin başında belirlenen sayıyla kısıtlanır.
- Bu sayı process'in türüne göre değişebilir.
- Statik yer atamada process'in izin verilen sayıda sayfası ana belleğe yüklendikten sonra, yeni sayfalarına ihtiyaç duyulursa, ilgili process'in eski sayfalarından bazıları ana bellekten çıkarılarak yer açılır.
- Bu yaklaşım yerel sayfa çıkarma politikası (local replacement policy) olarak bilinir.

Bellekte Çerçeve(frame) Ataması

- **Dinamik yer atama:**
- Dinamik yer atamada process'lerin işletimi boyunca tahsis edilen çerçeve (frame) sayısı değişiklik gösterir.
- Sayfa hatalarının (page fault) fazla olması durumunda ek çerçeve tahsisi, sayfa hatalarının az olması durumunda ise çerçeve azaltılması yapılır.

- Yerel sayfa çıkarma politikasının tersi, genel sayfa çıkarma politikasıdır (global replacement policy).
- Genel sayfa çıkarma politikasıyla, process'lere atanacak çerçeve sayısına bir kısıtlama konmaz.
- Bunun sonucu olarak, fiziksel bellekte yer açmak gerektiğinde sadece ilgili process değil diğer process'lerin sayfalarının da bellekten çıkarılması söz konusu edilir.

Bellekte Çerçeve(frame) Ataması



(a) Paging to static swap area. (b) Backing up pages dynamically.

Bellek Çerçevesi Atama Politikaları

- Sayfalı sanal bellek yönetiminin uygulandığı sistemlerde, process'lere ayrılan bellek çerçeve sayısı sistem başarımını etkileyen önemli bir parametre olarak ortaya çıkar.

- Bir process'in aynı anda sahip olabileceği toplam fiziksel çerçeve sayısının, genelde bir üst sınırı bulunur.

- Multiprocess sistemlerde fiziksel bellek, aynı anda birden çok process tarafından paylaşılmak zorunda olduğundan her bir process'e, fiziksel bellekteki sınırlı sayıdaki çerçeveden ancak bir bölümünün ayrılabilmesi doğaldır.

Bellek Çerçevesi Atama Politikaları

- Fiziksel bellekteki toplam çerçeve (frame) sayısının değişmez olduğu düşünüldüğünde (statik yer atama), her bir process için ana bellekte ayrılabilen çerçeve sayısı, sistemde aynı anda işletime alınan process sayısına bağlıdır.

- Daha açık bir anlatımla, process'lere ayrılan çerçeve sayısı, aynı anda işletime alınan process sayısıyla ters orantılıdır.

Bellek Çerçevesi Atama Politikaları

- Process'lere atanabilecek çerçeve sayısının bir üst bir de alt sınırı bulunur.

- *Process'lere ayrılan çerçeve sayısının çok düşük tutulması durumunda*, sayfa hatasının artışına bağlı olarak, fiziksel bellek-disk arası sayfa değişimlerinin ve işletim yükünün artmasıyla karşı karşıya kalınır.

Bellek Çerçevesi Atama Politikaları

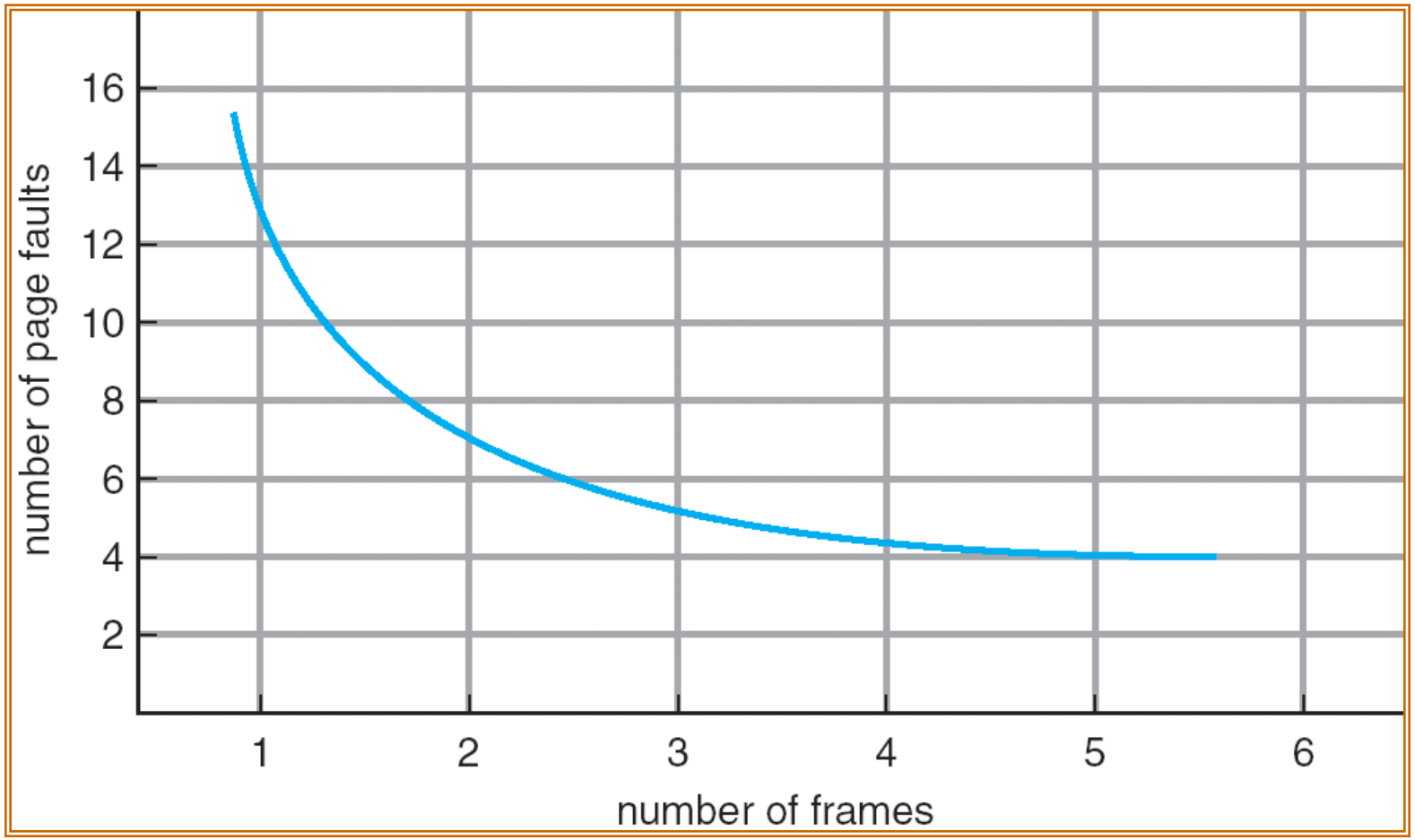
- Bu nedenle process'lere aynı anda atanabilecek toplam çerçeve sayısının, işletim sistemince dikkatlice belirlenmesi önem taşır.

- Bu belirleme, genelde sayfa hatası sayısı taban alınarak yapılır.

Bellek Çerçevesi Atama Politikaları

- Sistemde kaynak kullanım düzeyini artırmak amacıyla aynı anda yürütülen process sayısını artırmak gerekir.
- Ancak aynı anda yürütüme alınan process sayısı artıkça process'lerin her birine ayrılabilen çerçeve sayısı düşer.
- Bu düşüş belirli bir düzeyin altına indiğinde, birlikte çalışan process sayısının artmasına karşın kaynak kullanım düzeyi, özellikle de CPU kullanım düzeyi düşmeye başlar.
- Zira process'lere yeterli sayıda fiziksel bellek çerçevesi ayıramama sonucunda bellekte bulunmayan sayfalara erişim istemleri, dolayısıyla sayfa hataları sayıca artar.
- Böylece process'lerin CPU kullanım payı ve sistem başarımı aşırı düşer; işletim yükü artar.

Çerçeve Sayısı ve Sayfa Hatası Grafiği



Sayfa Değiştirme (Page replacement)

- Sayfa hatası (page fault) sonrasında, erişilmek istenen sayfanın yerleştirileceği boş bir çerçeve bulunamaması durumunda bellekten çıkarılacak kurban (victim) çerçeve seçimi gündeme gelir.
- Kurban (victim) çerçeve seçimi Sayfa değiştirme algoritmalarına (Page replacement algorithm) dayalı olarak yürütülür.
- Sayfa Değiştirme algoritmalarının hepsi, '**sayfa hatası (page fault)**' sayısını en aza indirmeyi amaçlar.
- Ancak bu algoritmaların işletim süreleri de göz önüne alınması gereken diğer bir kıstası oluşturur.

Sayfa Değiştirme Algoritmaları (Page replacement)

- Sayfa değiştirme algoritmalarının amacı hangi sayfanın fiziksel bellekte ve hangi sayfanın sanal bellekte bulunacağını ve ne zaman yer değiştireceklerini belirlemektir.

- Algoritmaların başarısı, seçilen kurban çerçeve içindeki sayfanın yakın gelecekte kullanılmaması ile belirlenebilir.

- Algoritmaların çoğu process'lerin geçmiş davranışlarına bakarak gelecek hakkında tahmin yürüterek çalışmaktadır.

Sayfa Değiştirme Algoritmaları (Page replacement)

1) En uygun (Optimal page replacement)

2) En önce kullanılan (Least recently used-LRU)

3) İlk-giren-ilk-çıkar (First-in-first-out-FIFO)

4) Saat (Clock)

En Uygun Algoritması (Optimal page replacement)

- Optimal sayfa değiştirme algoritmasına göre bir sayfa hatası (page fault) olduğunda,
- yani fiziksel bellekte bulunmayan, dolaysıyla sanal bellekteki bir sayfaya erişilmek istendiğinde,

- Sanal bellekten ilgili sayfa fiziksel belleğe yüklenirken, fiziksel bellekte bundan sonra en uzun süre erişilmeyecek olan çerçeve (victim frame) seçilir ve buraya sanal bellekten alınan ilgili sayfa yüklenir.
- Seçilen kurban çerçeve (victim frame) içeriği olan sayfa üzerinde değişiklik yapılmışsa sanal belleğe geri yazılır değişiklik yoksa sadece sayfa tablosundan çıkarılır.

En Uygun Algoritması (Optimal page replacement)

- Optimal sayfa değiştirme algoritması kuramsal olarak en iyi sonucu verecek (en az sayfa hatası oluşturan) algoritmadır.
- Zira ana bellekten bir çerçeve çıkarmak gerektiğinde, bu algoritmaya göre, **o an erişimi en geç gerçekleşecek çerçevenin seçimi** söz konusu edilecektir.
- Ancak herhangi bir anda hangi çerçevenin en geç erişilecek olacağını bilmenin ya da kestirebilmenin kolay ve pratik bir yolu yoktur.
- Bu nedenle Optimal sayfa değiştirme algoritması, daha çok kuramsal değerde, diğer algoritmalarla elde edilen sonuçların değerlendirilmesine yarayan ölçüt bir algoritma olarak algılanmalıdır.

En Uygun Algoritması (Optimal page replacement)

- **Örnek:**
- Bir process'in **beş sayfası (1,2,3,4,5)** ve bu sayfaların yerleşebileceği fiziksel ana bellekte bulunan **üç tane çerçeve (frame)** bulunduğunu varsayınız.
- Process'in sayfalarının aşağıdaki sırayla fiziksel ana belleğe taşınması gerekir.
-

	1	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*
Page reference string :	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**

En Uygun Algoritması (Optimal page replacement)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

1. İstek için, sayfa hatası (page fault) oluşur çünkü 2. sayfa henüz bellekte olmayan bir sayfadır.

|2| | |

2. İstek için, sayfa hatası olur çünkü 3. sayfa da henüz bellekte bulunmamaktadır.

|2|3| |

3. İstek için, zaten bellekte bulunan 2. sayfa istenir ve bir sayfa hatası oluşmaz.

|2|3| |

4. İstek için, yeniden bir sayfa hatası oluşur, sebebi 1. sayfanın da henüz bellekte olmamasıdır.

|2|3|1|

En Uygun Algoritması (Optimal page replacement)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- Artık ana bellek çerçeveleri tamamen dolmuştur çünkü bellek kapasitesi 3 sayfa taşıyacak şekilde 3 çerçeve ile tanımlanmıştır.

- 5. istekte, bellekte bulunmayan 5 numaralı sayfa talep edilir ve bir sayfa hatası oluşur. Bu noktada Optimal Sayfa Değiştirme algoritması devreye girer. Çünkü bellek tamamen doludur ve 5 numaralı sayfanın belleğe yüklenmesi için bellekteki çerçevelerden birisinin sanal belleğe gönderilmesi gerekmektedir. Sorun hangi çerçevenin içindeki sayfanın boşaltılması gerektiğidir.

En Uygun Algoritması (Optimal page replacement)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- |2|3|1|
- Optimal sayfa değiştirme algoritmasına göre en uzun süre erişilmeyecek olan çerçevedeki sayfa kaldırılır.
- 5. istekte, **1 numaralı sayfaya** çalışma sonuna kadar erişilmeyeceği için bu sayfa kaldırılacak ve yerine 5 numaralı sayfa konulacaktır.
- |2|3|5|

En Uygun Algoritması (Optimal page replacement)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 6. istekte, zaten bellekte bulunan **2. sayfa istenir** ve bir sayfa hatası oluşmaz.
- |2|3|5|
- 7. istekte, bellekte bulunmayan **4 numaralı sayfa talep edilir** ve bir sayfa hatası oluşur. Bellek tamamen doludur ve 4 numaralı sayfanın belleğe yüklenmesi için bellekteki çerçevelerden birisinin içindeki sayfanın kaldırılması gerekmektedir. En uzun süre erişilmeyecek olan **2. sayfa kaldırılır** ve yerine 4 numaralı sayfa konulacaktır.
- |4|3|5|

En Uygun Algoritması (Optimal page replacement)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 8. istekte, zaten bellekte bulunan **5. sayfa istenir** ve bir sayfa hatası oluşmaz.
- |4|3|5|
- 9. istekte, zaten bellekte bulunan **3. sayfa istenir** ve bir sayfa hatası oluşmaz.
- |4|3|5|

En Uygun Algoritması (Optimal page replacement)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 10. istekte, bellekte bulunmayan **2 numaralı sayfa** talep edilir ve bir sayfa hatası oluşur. Bellek tamamen doludur ve 2 numaralı sayfanın belleğe yüklenmesi için bellekteki çerçevelerden birinin içindeki sayfanın kaldırılması gerekmektedir. En uzun süre erişilmeyecek olan **4. sayfa kaldırılır** ve yerine 2 numaralı sayfa konulacaktır.
- |2|3|5|
- 11. istekte, zaten bellekte bulunan 5. sayfa istenir ve bir sayfa hatası oluşmaz.
- |2|3|5|
- 12. istekte, zaten bellekte bulunan 2. sayfa istenir ve bir sayfa hatası oluşmaz.
- |2|3|5|

En Uygun Algoritması (Optimal page replacement)

- Bu algoritmada, **gelecekte istenilebilecek en son sayfa seçilir** ve sanal belleğe taşınır.
- Toplam 6 tane sayfa hatası oluştu.

Ana belleğe taşınacak sayfalar	2		3		2		1		5		2		4		5		3		2		5		2
Çerçeve 1	2		2		2		2		2		2		4		4		4		2		2		2
Çerçeve 2			3		3		3		3		3		3		3		3		3		3		3
Çerçeve 3							1		5		5		5		5		5		5		5		5
Sayfa hatası	X		X				X		X				X						X				

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

- Bu algoritmaya göre bir sayfa hatası (page fault) olduğunda, yani fiziksel bellekte bulunmayan bir sayfaya erişilmek istendiğinde, dolaysıyla sanal bellekteki bir sayfaya erişilmek istendiğinde,
- Sanal bellekten ilgili sayfa fiziksel belleğe yüklenirken, bellekteki en az erişilen sayfanın (yakın geçmişte kullanılmamış) çerçevesi kullanılır.

- **Bu algoritmada ana bellekteki en önce kullanılmış olan sayfa değiştirilir.**
- En uygun algoritması kadar iyi bir performansı vardır.
- Ancak uygulanması zor bir algoritmadır.

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

- En önce kullanılan algoritması sayfaya erişilme zamanını taban alır.
- Bu algoritma son zaman diliminde sıkça kullanılan sayfaların gelecek zaman diliminde de sıkça kullanılacağını ya da son zaman diliminde hiç kullanılmamış bir sayfanın gelecek zaman diliminde de kullanılma olasılığının düşük olacağını varsayar.
- Bu varsayıma göre bellekten bir sayfa çıkarılacağı zaman o ana kadar, en erken erişilmiş sayfanın seçimi yapılır.
- Ancak en erken erişilmiş sayfayı belirlemek kolay bir işlem değildir ve çoğu kez özel donanım desteği gerektirir.

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

- **Örnek:**
- Bir process'in beş sayfası (1,2,3,4,5) ve bu sayfaların yerleşebileceği fiziksel ana bellekte bulunan üç tane çerçeve (frame) bulunduğunu varsayınız.
- Process'in sayfalarının aşağıdaki sırayla fiziksel ana belleğe taşınması gerekir.

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 1. istekte, sayfa hatası (page fault) olur çünkü henüz bellekte olmayan 2. sayfa talep edilmiştir.
- |2| | |
- 2. istekte, sayfa hatası (page fault) olur çünkü henüz bellekte olmayan 3. sayfa talep edilmiştir.
- |2|3| |
- 3. istekte, zaten bellekte bulunan 2. sayfa istenir ve bir sayfa hatası oluşmaz.
- |2|3| |
- 4. istekte, sayfa hatası (page fault) olur çünkü henüz bellekte olmayan 1. sayfa talep edilmiştir.
- |2|3|1|

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- artık bellek tamamen dolmuştur çünkü bellek kapasitesi 3 sayfa taşıyacak şekilde tanımlanmıştır.
- 5. istekte, bellekte bulunmayan 5 numaralı sayfa talep edilir ve bir sayfa hatası oluşur. Bu noktada En Önce Kullanılan (Least recently used) algoritması devreye girer. Çünkü bellek tamamen doludur ve 5 numaralı sayfanın belleğe yüklenmesi için bellekteki sayfalardan birisinin kaldırılması gerekmektedir. Sorun hangi sayfanın kaldırılacağıdır.

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- LRU algoritmasına göre en eski erişilen kaldırılır. Buna göre en son erişilen iki sayfa, **2. ve 1.** sayfalardır. Dolayısıyla **3. sayfa**, şu anda en eski erişilmiş olandır ve bellekten kaldırılır.
- |2|5|1|

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 6. istekte, zaten bellekte bulunan 2. sayfa istenir ve bir sayfa hatası oluşmaz.
- |2|5|1|
- 7. istekte, bellekte bulunmayan 4 numaralı sayfa talep edilir ve bir sayfa hatası oluşur. Bellek tamamen doludur ve 4 numaralı sayfanın belleğe yüklenmesi için bellekteki sayfalardan birisinin kaldırılması gerekmektedir. **1. sayfa**, şu anda en eski erişilmiş olandır ve bellekten kaldırılır ve yerine **4. sayfa** yerleştirilir.
- |2|5|4|

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 8. istekte, zaten bellekte bulunan 5. sayfa istenir ve bir sayfa hatası oluşmaz.
- |2|5|4|
- 9. istekte, bellekte bulunmayan 3. sayfa talep edilir ve bir sayfa hatası oluşur. **2. sayfa**, şu anda en eski erişilmiş olandır ve bellekten kaldırılır ve yerine **3. sayfa** yerleştirilir.
- |3|5|4|

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

- 10. istekte, bellekte bulunmayan 2. sayfa talep edilir ve bir sayfa hatası oluşur. **4. sayfa**, şu anda en eski erişilmiş olandır ve bellekten kaldırılır ve yerine **2. sayfa** yerleştirilir.
- |3|5|2|
- 11. istekte, zaten bellekte bulunan 5. sayfa istenir ve bir sayfa hatası oluşmaz.
- |3|5|2|
- 12. istekte zaten bellekte bulunan 2. sayfa istenir ve bir sayfa hatası oluşmaz.
- |3|5|2|

En Önce Kullanılan Algoritması (Least Recently Used-LRU)

- Bu algoritmada ana bellekteki en önce kullanılmış olan sayfa değiştirilir.
- Toplam 7 tane sayfa hatası oluştu.

Ana belleğe taşınacak sayfalar	2		3		2		1		5		2		4		5		3		2		5		2
Çerçeve 1	2		2		2		2		2		2		2		2		3		3		3		3
Çerçeve 2			3		3		3		5		5		5		5		5		5		5		5
Çerçeve 3							1		1		1		4		4		4		2		2		2
Sayfa hatası	x		x				x		x				x				x		x				

İlk-Giren-İlk-Çıkar Algoritması (First-in-first-out-FIFO)

- Bu algoritma, **ana bellekte uzun süre kalan bir sayfanın**, gelecek komut işletimlerinde erişilme olasılığının düşük olacağı varsayımına dayanır.
- Bu algoritmanın uygulandığı sistemlerde, process'in ana belleğe yüklü tüm sayfaları ilk giren ilk çıkar (FIFO) türü bir kuyruk yapısında tutulur.
- Bu kuyruk yeni bir sayfaya erişmek gerektiğinde güncellenir.
- Kuyruğun başında belleğe ilk yüklenen, sonunda da en son yüklenen process sayfası yer alır.
- Bellekten bir sayfa çıkarmak gerektiğinde kuyruk başındaki sayfa seçilir.
- Bu algoritmada, FIFO kuyruğu, sadece erişilen sayfa değiştiğinde güncellenir.

İlk-Giren-İlk-Çıkar Algoritması (First-in-first-out-FIFO)

- Uygulanması kolay, işletim yükü düşük, hızlı bir algoritmadır.
- **Ana bellekte en uzun süredir bulunan sayfa değiştirilir.**
- Bellekte uzun süre kalmış sayfalar, bu algoritmanın varsaydığı gibi, her zaman, gelecek komut işletimlerinde kullanım olasılığı mutlaka en düşük olan sayfaları oluşturmazlar.
- Process'in en sık kullanılan sayfaları da belleğe en erken girmiş sayfalar olabilir. Bu durumda algoritma, sık sık başvurulan sayfaları bellekten çıkararak fiziksel bellek-disk arası sayfa gelgitlerini artıran bir algoritma olarak ortaya çıkar.

İlk-Giren-İlk-Çıkar Algoritması (First-in-first-out-FIFO)

- **Örnek:**
- Bir process'in beş sayfası (1,2,3,4,5) ve bu sayfaların yerleşebileceği fiziksel ana bellekte bulunan üç tane çerçeve (frame) bulunduğunu varsayınız.
- Process'in sayfalarının aşağıdaki sırayla fiziksel ana belleğe taşınması gerekir.

Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

İlk-Giren-İlk-Çıkar Algoritması (First-in-first-out-FIFO)

- **Örnek Çözümü:**
- Ana bellekte en uzun süredir bulunan sayfa değiştirilir.
- Toplam 9 tane sayfa hatası oluştu.

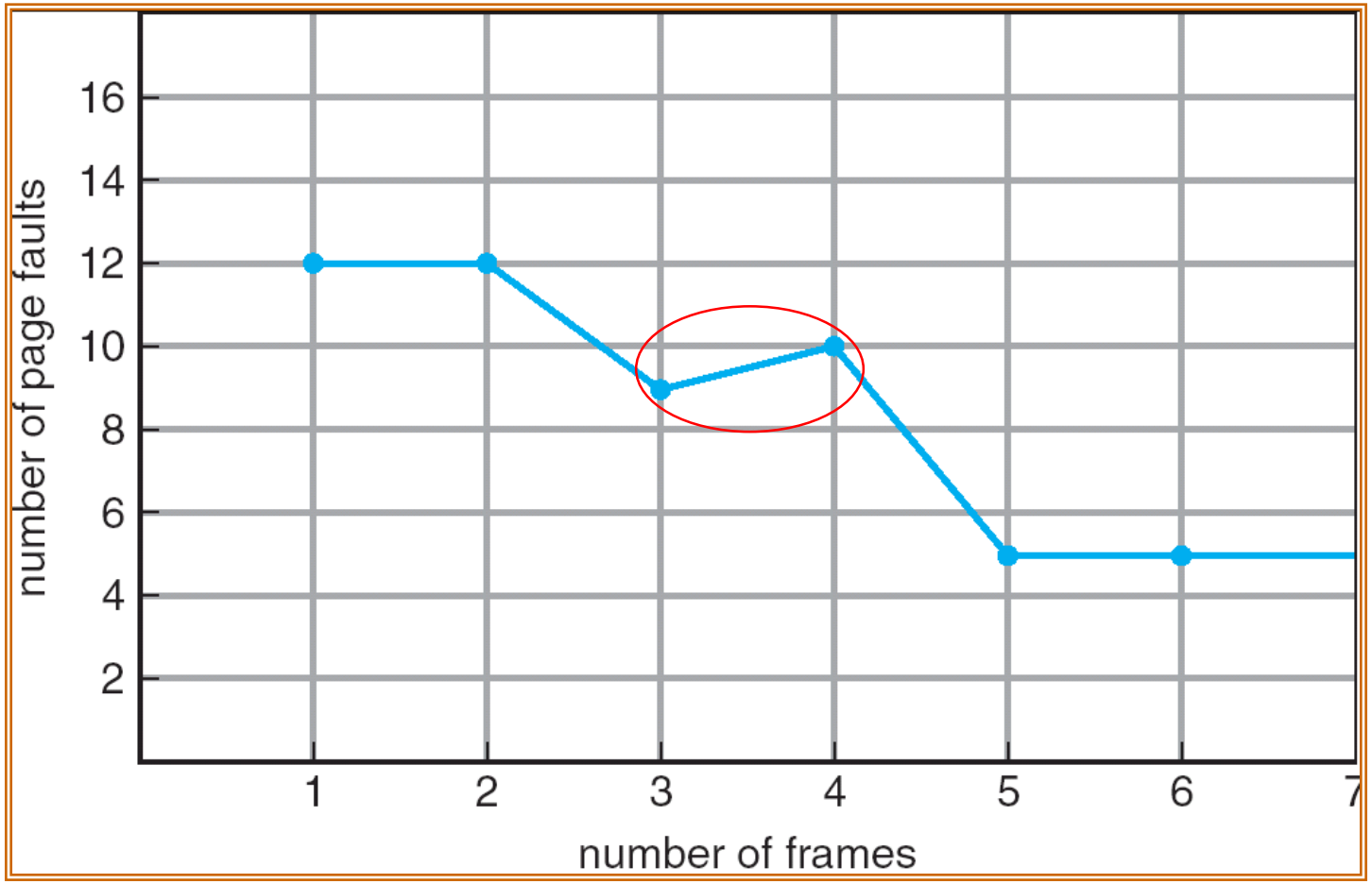
Page reference string : 2 3 2 1 5 2 4 5 3 2 5 2

Ana belleğe taşınacak sayfalar	2		3		2		1		5		2		4		5		3		2		5		2
Çerçeve 1	2		2		2		2		5		5		5		5		3		3		3		3
Çerçeve 2			3		3		3		3		2		2		2		2		2		5		5
Çerçeve 3							1		1		1		4		4		4		4		4		2
Sayfa hatası	x		x				x		x		x		x				x				x		x

Belady Anormalliği

- Bellekte bir process'e ayrılan boş çerçeve sayısı ne kadar fazla olursa bu process'in üreteceği sayfa hatası (page fault) uyarısı sayısının da o denli küçük olacağını düşünmek çok doğaldır.
- Ancak, kimi zaman, process'e ayrılan boş çerçeve sayısının artırılmasına karşın sayfa hatası sayısının da arttığı görülür.
- Ters orantılı olması gereken boş çerçeve ve sayfa hatası sayılarının doğru orantılı olarak gelişmesi normal değildir.
- Bu durum **Belady Anormalliği** olarak adlandırılır.
- Bunun, process sayfalarının ilk giren ilk çıkar türü bir veri yapısı içinde tutulmasından kaynaklandığı bilinir.
- Bu anomaliden etkilenen sayfa değiştirme algoritmaları arasında FIFO (First in First out), LRU (Least Recently Used) ve LFU (Least Frequently Used) gibi algoritmalar bulunmaktadır.

Belady Anormalliği



Belady Anormalliği

zaman →

Belleğe taşınacak sayfalar	0	1	2	3	0	1	4	0	1	2	3	4
Fiziksel bellek (**3 çerçeve**)	0	0	0	1	2	3	0	0	0	1	4	4
		1	1	2	3	0	1	1	1	4	2	2
			2	3	0	1	4	4	4	2	3	3

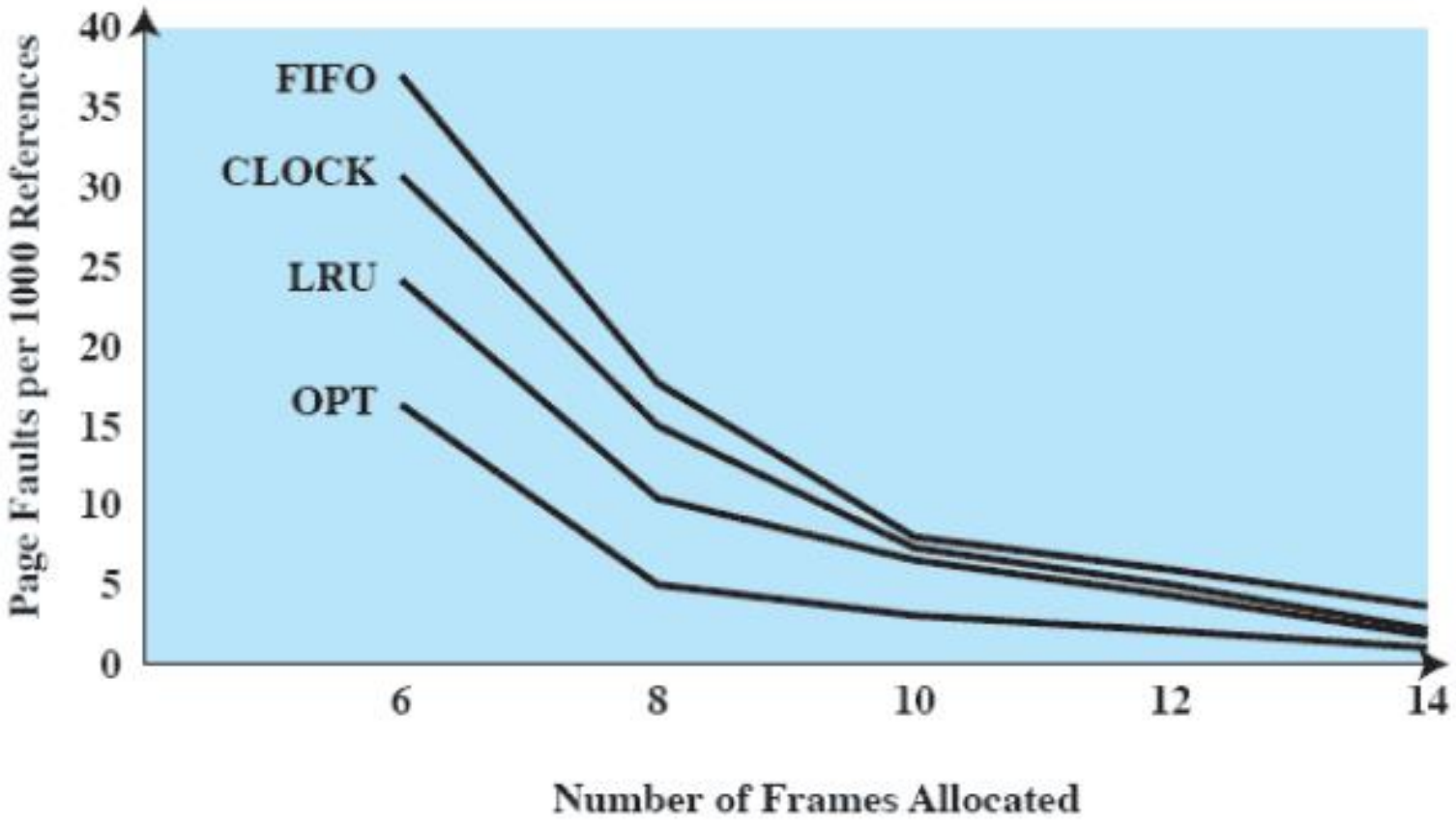
9 sayfa hatası!

zaman →

Belleğe taşınacak sayfalar	0	1	2	3	0	1	4	0	1	2	3	4
Fiziksel bellek (**4 çerçeve**)	0	0	0	0	0	0	1	2	3	4	0	1
		1	1	1	1	1	2	3	4	0	1	2
			2	2	2	2	3	4	0	1	2	3
				3	3	3	4	0	1	2	3	4

10 sayfa hatası!

Algoritmaların Karşılaştırılması



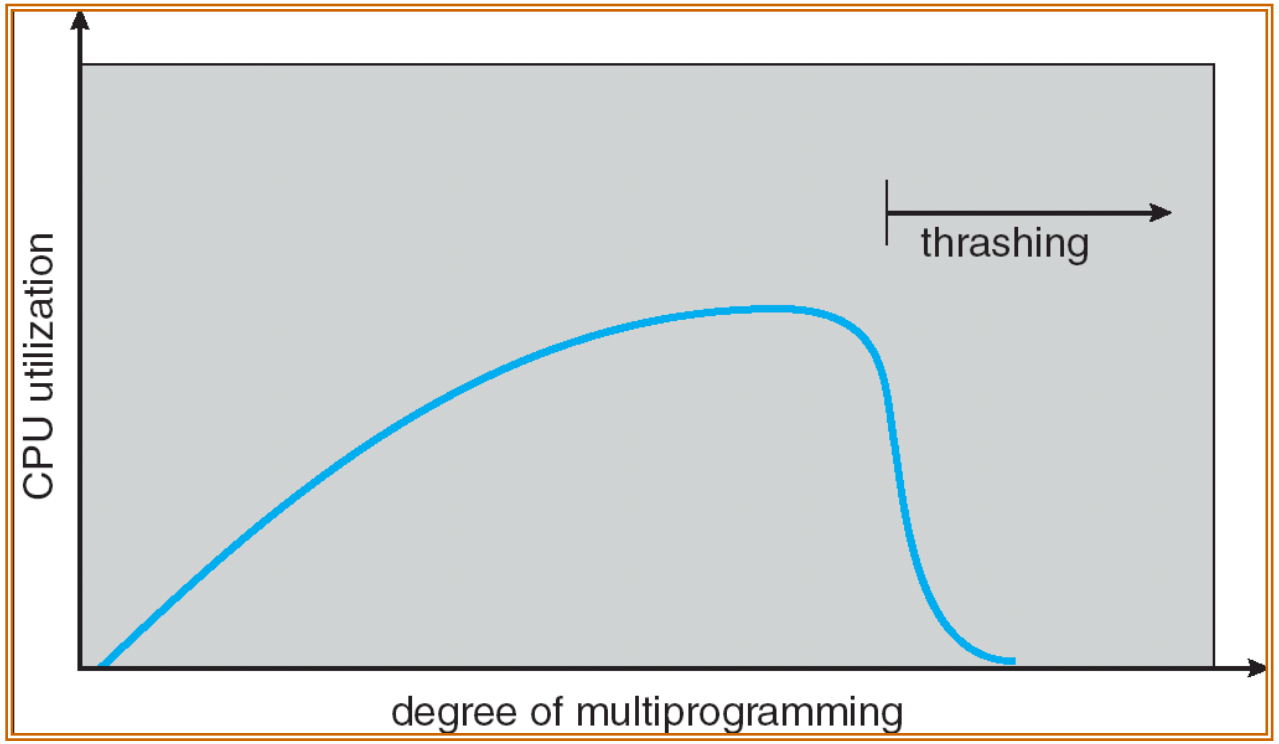
Boşuna Çalışma (Thrashing)

- Bir process için gerekli minimum bellek çerçeve sayısının altına düşüldüğünde, process beklemeye alınır.
- Bu process'in tüm çerçeveleri boşaltılır ve diğer process'lere tahsis edilir.
- Bir process yeterli sayıda çerçeveye sahip değilse sıklıkla sayfa hatası olur.
- Sayfa hatası olduğunda process'e ayrılan çerçevelerden birisi kurban (victim) olarak seçilir ve yer değiştirilir.
- Ardından process diğer sayfaya ihtiyaç duyar ve yeniden sayfa hatası olur.
- Sayfa hatası sıklığı giderek artar.
- Yüksek orandaki sayfalama işlemine **boşuna çalışma (thrashing)** denilir.

Boşuna Çalışma (Thrashing)

- “Thrashing” aşırı disk kullanımına neden olur.
- Thrashing oluştuğunda, alarm verdirici bir oranda sayfa hatası oluşur; işletim sistemi zamanının çoğunu, sayfaları sanal bellek ile disk arasında götürüp getirmeye ayırmak zorunda kalır.
- Thrashing kısa zamanda sistem kaynaklarının tükenmesine ve genelde de cevap süresinin uzamasına neden olur.
- Eğer çalışmakta olan uygulama artık cevap vermiyorsa ama disk sürücünün LED’i yanıp sönüyorsa bilgisayar büyük olasılıkla thrashing kurbanı olmuştur.

Boşuna Çalışma (Thrashing)



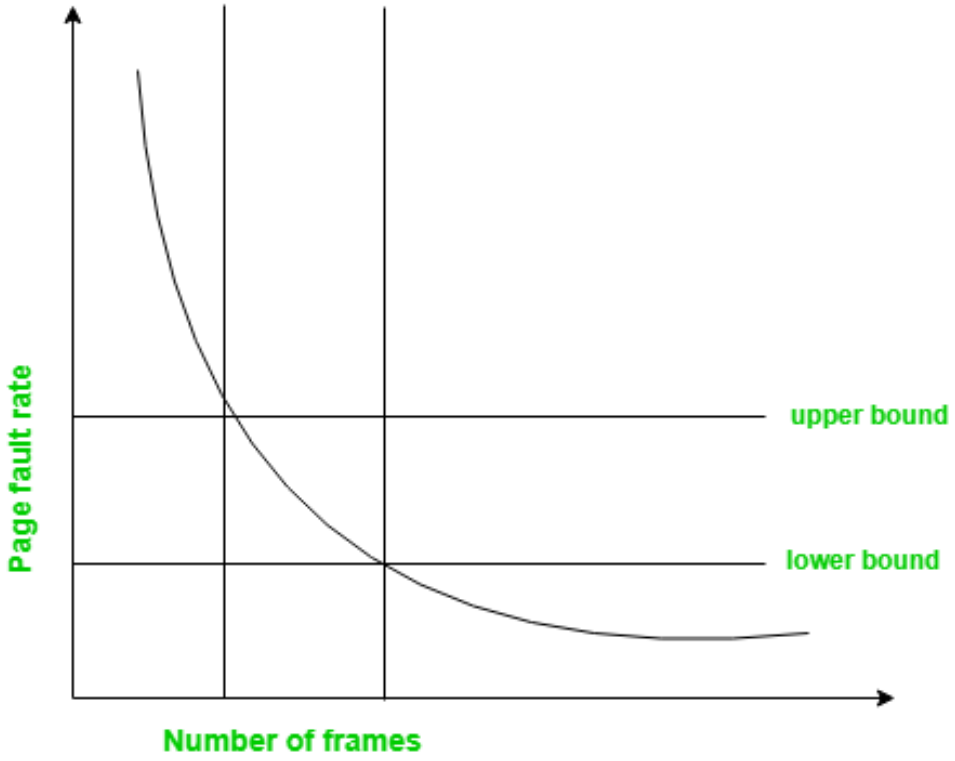
CPU performansı, multiprocess seviyesi arttıkça artmakta, fakat maksimum performansına eriştikten sonra multiprocess seviyesi arttıkça mikroişlemci performansı birden düşmektedir.

Bu ani düşüşün olduğu kısım thrashing'in (etkinlik kaybı) başladığı noktadır.
Bu noktada CPU performansını artırmak için multiprocess seviyesi azaltılmalıdır.

Boşuna Çalışma (Thrashing)

- Sayfa hata oranı çok yüksekse, process'in kendisine bellekte atanmış çok az çerçeve (frame) olduğunu gösterir.
- Düşük sayfa hata oranı ise, process'in çok fazla çerçeveye (frames) sahip olduğunu gösterir.

- Thrashing ile ilgili sorun yüksek sayfa hatası oranıdır bu nedenle, sayfa hata oranı kontrol altında tutulmalıdır.



Boşuna Çalışma (Thrashing)

- İşletim sistemi CPU doluluk oranını sürekli izler ve CPU doluluk oranı azaldığında multiprocess seviyesini artırarak yeni process başlatır.

- Global page replacement kullanılırsa, diğer process'lerin sayfaları alınmaya başlar ve buna bağlı olarak sayfa hatası sıklığı artmaya başlar.

- Diğer process'ler yeni sayfaya ihtiyaç duyduğunda bu kez yeni process'e atanan sayfalar alınır.

Boşuna Çalışma (Thrashing)

- Sayfa hatası arttıkça, process'ler sayfalama biriminde sıra beklemeye başlar ve CPU hazır kuyruğu boşalır. CPU doluluk oranı düşmeye başlar.

- CPU process zamanlayıcı (scheduler), CPU'nun doluluk oranının düşmeye başladığını görür ve sürekli yeni process'ler başlatır.

- Sonuçta multiprocess seviyesi artar, ancak CPU doluluk oranı düşer, thrashing artar ve throughput düşer.